
Large Language Models with At Most One Spike per Neuron

Zhuoya Zhao
zoezhao0427@gmail.com

Parsa Omid
parsa.omidi1@huawei.com

Aref Jafari

Richard Naud
rnaud@uottawa.ca

Abstract

Leveraging their inherent sparse event-driven computation, spiking neural networks (SNNs) offer a promising path toward energy-efficient large language models (LLMs). Time-to-first-spike (TTFS) coding generates at most one spike per neuron within a time window, yielding extremely low firing rates. However, conventional TTFS SNNs are restricted to specific structures, making it challenging to encode certain blocks in LLMs—such as layer normalization and matrix multiplications—using TTFS. To overcome this limitation, we introduce a reference-based strategy specifically to encode the four core LLM components: embedding layers, layer normalization, attention-related operations and dropout. We construct a fully TTFS-based SNN architecture and train it end-to-end. Experiments on modern LLMs like BERT and GPT-2 demonstrate that our approach achieves performance comparable to ANN counterparts on natural language understanding and common-sense reasoning, while a clear gap remains on language modeling perplexity. To the best of our knowledge, this is the first work to scale a spiking LLM to 1.5 billion parameters using TTFS coding. We also report an estimate of spike-related energy; this is a spike-count proxy under an established cost model rather than a measurement on neuromorphic hardware.

1 Introduction

As an alternative to conventional ANN-based computation, spiking neural networks (SNNs) have gained increasing attention because their event-driven processing relies on sparse and binary spikes, which makes them naturally compatible with energy-efficient neuromorphic hardware. In particular, neuromorphic chips designed for spiking computation leverage in-memory computing to avoid frequent data transfer between memory and processors, thereby accelerating computation Merolla et al. [2014]. This characteristic of energy efficiency gives SNNs the potential to scale large language models (LLMs) for practical implementation.

Compared to the time-average rate code Gerstner et al. [2014], the population average rate code DePasquale et al. [2023] and the temporal firing patterns Friedenberger et al. [2023], Time-to-First-Spike (TTFS) coding encodes information using strictly at most one spike per neuron, yet it enables SNNs to maintain high performance by leveraging precise spike timing Stanojevic et al. [2024]. Due to the sparsity and energy efficiency of TTFS coding, recent microelectronic research has demonstrated its computational advantages and developed dedicated mixed-signal vector-by-matrix multiplication (VMM) and neuron circuits that naturally align with the algorithmic principles of TTFS Bavandpour et al. [2019], Widmer et al. [2023], opening up broad prospects for the application of TTFS in edge intelligence. Despite its inherent sparsity, TTFS coding faces significant challenges when applied to LLMs. Prior works have demonstrated that TTFS can establish an exact mapping for

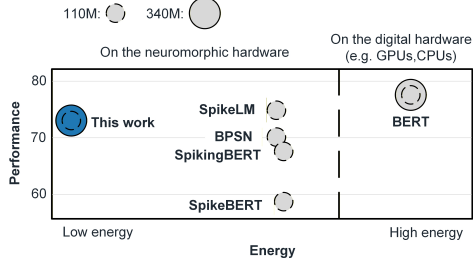


Figure 1: Comparison of performance, estimated spike-related energy consumption (refer to Section 6.1), and parameter size for TTFS-BERT, standard BERT, and other spiking BERTs on the GLUE benchmark.

a specific class of artificial neural networks, namely Multi-Layer Perceptrons (MLPs) with Rectified Linear Units (ReLU)s Rueckauer and Liu [2018], Stanojevic et al. [2023, 2024]. The first spike timing is not a continuous value, since the output spike timing is delayed relative to the input, and any infinite first spike timing is usually disregarded. Consequently, the first spike timing is inherently upper-bounded, and if it exceeds this limit, it is clipped to the predefined maximum value. This constitutes the first major limitation of TTFS: the restricted range of output spike timings. Moreover, because TTFS mapping is only feasible for specific network structures, it is challenging to implement more complex components Stanojevic et al. [2024], such as embedding layers, layer normalization (LayerNorm), and matrix multiplication, using conventional TTFS neurons.

Recent work has explored latency coding for converting ANNs into spiking transformers Jiang et al. [2024], Zhao et al. [2025]. These conversion-based approaches, may suffer from approximation errors, potentially impacting inference accuracy. To address this, we advocate for designing ANNs with inherent SNN-friendly properties—such as spike-compatible activations, which significantly enhances both accuracy and hardware efficiency. Consequently, we design a native TTFS-based LLM and train it from scratch. Our main idea is to construct an exact mapping between TTFS layers and certain components in LLMs (e.g., linear layers). For components that do not admit an exact mapping, we instead approximate them using TTFS layers (e.g., LayerNorm). Evaluating this architecture requires rigorous consideration of training dynamics, particularly as we scale from small networks to LLMs. To validate the scalability of our approach, we conduct comprehensive experiments on LLMs across varying parameter scales.

Scaling SNNs for computational purposes ultimately targets their deployment on neuromorphic chips. Therefore, two key questions must be addressed: first, how to realize TTFS-based LLMs; and second, how to maintain competitive performance comparable to their ANN version. To the best of our knowledge, this is one of the first works to explore TTFS coding for large-scale language models. Our main contributions are summarized as follows:

- (1) We introduce a reference-time mechanism in LLMs that enhances the expressiveness of TTFS neurons, enabling signed activations.
- (2) We extend reference-based TTFS (R-TTFS) encoding to four key modules (embedding layers, LayerNorm, matrix multiplication, dropout) in LLMs, enabling TTFS-based computation throughout the entire LLM pipeline.
- (3) We show that our method scales effectively to LLMs such as BERT and GPT-2, reaching ANN-level performance on natural language understanding and commonsense reasoning benchmarks.

All these contributions point toward spiking LLMs whose spike count, and hence estimated spike-related energy, is substantially lower than that of rate-coded spiking baselines (Figure 1).

2 Related works

2.1 Neural Coding Schemes

There are four common coding schemes in SNNs: time-average rate code, population average rate code DePasquale et al. [2023], temporal firing patterns Friedenberger et al. [2023] and latency coding

(e.g. TTFS). From the perspective of time-average rate code, the fundamental unit of information and computation is the firing rate, which experimentally measures a finite number of spikes within a limited time window Gerstner et al. [2014]. To reduce the required counting time, populations of neurons with noise can estimate the firing rate over a shorter time window. Employing population coding requires additional memory and poses challenges for scaling up. Burst coding is a neural coding scheme in which information is represented by a rapid sequence of spikes rather than by single spikes or average firing rates. Alternatively, temporal coding identifies precise spike timing as the fundamental computational unit. TTFS uses the timing of the first spike to encode information, utilizing at most one spike, making it a very sparse coding scheme. The main advantage of TTFS are energy efficiency Davies et al. [2021] driven by low firing rates.

2.2 Evolution and Principles of TTFS Coding

The basic idea of TTFS is to only consider the time of the first threshold crossing Gerstner and Kistler [2002]. Absolute and relative spike timings are two approaches for implementing TTFS coding in SNNs. In the absolute scheme, the actual spike timing is taken as the output of each layer Mostafa [2017], Göltz et al. [2021], whereas in the relative scheme, the output is defined as the difference between the spike timing and a reference time Rueckauer and Liu [2018], Stanojevic et al. [2023], Wei et al. [2023], Stanojevic et al. [2024], Zhang et al. [2025], Zhao et al. [2025]. Because the relative scheme operates within a fixed time range, while the absolute spike timing accumulates across layers, the relative scheme is more stable for scaling up neural networks.

2.3 Challenges in TTFS-based LLMs

Conventional TTFS coding has been limited to non-negative outputs, which prevents its direct application to common LLM components like LayerNorm and matrix multiplications. Consequently, it cannot directly satisfy the signed representational requirements of these modules. One feasible approach is to use an alternative reference time in place of the upper-bound time in order to generate negative values Zhao et al. [2025]. To approximate the complex operations, Jiang et al. proposed a method to compute high-dimensional operations by unrolling them into several steps Jiang et al. [2024]. However, they ignored the temporal relationship between spikes and did not explain how to use spike-related variables to represent the mean and variance. Although these variables can be interpreted in terms of firing rates, the corresponding spike trains are not explicitly available. To ensure compatibility with neuromorphic hardware, it is essential that these variables be represented directly through spike-based encoding. Another method is to modify the differential function of TTFS neurons so that, after integration, the differentiable function can match the nonlinearities used in ANNs Zhao et al. [2025]. Complex differential functions are difficult to realize precisely on the neuromorphic chips because their nonlinearities rely on Resistor–Capacitor (RC) circuits, Bipolar Junction Transistors, or memory resistor characteristics, whose parameters are intrinsic and hard to adjust Garg et al. [2024]. Researches on SNNs need to focus not only on theoretical aspects of algorithms and architectures but also on practical implementations on neuromorphic chips.

Implementing matrix multiplication using TTFS is challenging. One approach converts spike timing multiplication into logarithmic addition Zhao et al. [2025], but computing logarithms on neuromorphic hardware Davies et al. [2021], Pehle et al. [2022] incurs higher energy overhead due to multi-step approximations and memory accesses. Another method leverages membrane potentials and temporal splitting to accumulate multiplication results Jiang et al. [2024], but it is prone to estimation errors. These limitations motivate the development of more accurate and energy-efficient TTFS-based matrix multiplication schemes.

3 Preliminary

3.1 Time-to-first-spike model

TTFS is a case of latency coding and it considers the precise latency between the beginning of a stimulus and the first spike emitted by a neuron Rueckauer and Liu [2018], Göltz et al. [2021], Stanojevic et al. [2023, 2024]. The TTFS neuron dynamics are described by a piece-wise function comprising two phases: input accumulation and threshold crossing (Equation 1), which governs how the potential $V_i^{(n)}$ evolves over time t . $A_i^{(n)}$, $B_i^{(n)}$ and τ_c are hyperparameters and $W_{ij}^{(n)}$ is a

trainable parameter. H is the Heaviside step function. And $t_{min}^{(n)}$ is equal to $t_{max}^{(n-1)}$.

$$\tau_c \frac{dV_i^{(n)}}{dt} = \begin{cases} A_i^{(n)} + \sum_j W_{ij}^{(n)} H(t - t_j^{(n-1)}), & \text{for } t < t_{min}^{(n)} \\ B_i^{(n)}, & \text{for } t_{min}^{(n)} \leq t \leq t_{max}^{(n)} \end{cases} \quad (1)$$

As illustrated Figure 2A and 2B, the spike timings $t_1^{(n-1)}, t_2^{(n-1)}, t_3^{(n-1)}$ from the previous layer $(n-1)$ drive integrate-and-fire dynamics that generates the membrane potential $V_i^{(n)}$, shaped by a linear synaptic kernel. This dynamic process ensures that at most one spike occurs within the time window $[t_{min}^{(n)}, t_{max}^{(n)}]$. The membrane potential is reset when a spike occurs or $t \geq t_{max}$.

We consider the special case with $A_i^{(n)} = 0$ and $B_i^{(n)} = 1$, commonly referred to as the **B1-model**. By taking the integral of the neuron dynamics, the fundamental relationship between the input $t_j^{(n-1)}$ (stimulus timing) and the output $t_i^{(n)}$ (first-spike timing) of a linear layer is derived over the interval $t_{min}^{(n)} \leq t_i^{(n)} \leq t_{max}^{(n)}$, where θ denotes the firing threshold (Equation 2), enabling an exact mapping from ReLU-based ANNs to TTFS SNNs Stanojevic et al. [2024].

4 Methods

As outlined in the Preliminary, TTFS SNNs are exactly mapped from ReLU-based ANNs. However, this formulation imposes strict constraints on the output range and model architectures, as the output timing is inherently bounded by $t_{min}^{(n)}$ and $t_{max}^{(n)}$, and the integration dynamics are strictly linear with ReLU.

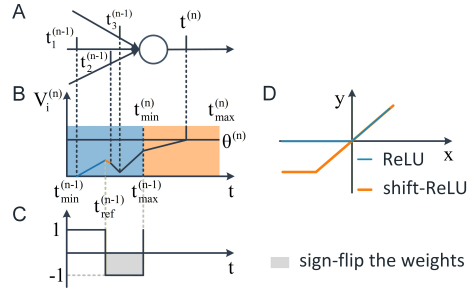


Figure 2: A. Spikes propagation. B. Integral of membrane potential $V_i^{(n)}$ over time. The blue background represents $t \in (t_{min}^{(n-1)}, t_{max}^{(n-1)})$, and the orange background represents $t \in (t_{min}^{(n)}, t_{max}^{(n)})$. The color change (from blue to orange) of the slope corresponds to the variation of the gates. C. Gating signal. D. ReLU and its variant.

Specifically, the architecture of LLMs extends beyond simple ReLU activations to incorporate a diverse set of components, including linear projections, LayerNorm, and attention mechanisms, all of which are essential to their overall behavior. However, conventional TTFS SNNs fail to address these complex architectural elements Stanojevic et al. [2024].

Therefore, in this section, first, we extend the B1-model to a R-TTFS variant and establish it as the fundamental building block for TTFS SNNs. We simulate the four main components—embedding layer, layer normalization, attention mechanism, and dropout—using the R-TTFS neuron.

4.1 Reference-based TTFS neuron

4.1.1 Expanding output range via reference time

To encode negative values via relative temporal shifts, we introduce a reference time t_{ref} to TTFS (Figure 2B).

$$t_{min}^{(n)} - t_i^{(n)} = \sum_{j=1}^N W_{ij}^{(n)} (t_{max}^{(n-1)} - t_j^{(n-1)}) - \tau_c \theta_i^{(n)} \quad (2)$$

To enable signed inputs and outputs, we set the reference point to the midpoint between $t_{\min}$ and $t_{\max}$, yielding Equation 3, where $\Delta = \frac{1}{2}(t_{\max} - t_{\min})$.

$$t_{\text{ref}}^{(n)} - t_i^{(n)} = \sum_{j=1}^N W_{ij}^{(n)} (t_{\text{ref}}^{(n-1)} - t_j^{(n-1)}) + \sum_{j=1}^N W_{ij}^{(n)} \Delta + \Delta - \tau_c \theta_i^{(n)} \quad (3)$$

4.1.2 Exact mapping via the linear regime of shift-ReLU

To derive the mapping relationship between R-TTFS SNNs and ANNs, we introduce shift-ReLU, which corresponds to a standard ReLU shifted negatively along $y = x$ as shown in Figure 2D. Because shift-ReLU allows both positive and negative inputs and outputs in its linear regime, the exact mapping between R-TTFS SNNs and linear layers of ANNs ($y = w_{ij}^{(n)} x + b_i^{(n)}$) still holds within this regime. For the given the weights $W_{ij}^{(n)}$ and the threshold $\theta_i^{(n)}$, the corresponding weight matrices $w_{ij}^{(n)}$ and bias vectors $b_i^{(n)}$ in the equivalent linear regime of shift-ReLU network can be expressed as follows:

$$\begin{aligned} w_{ij}^{(n)} &\stackrel{\text{def}}{=} W_{ij}^{(n)}, \\ b_i^{(n)} &\stackrel{\text{def}}{=} \sum_{j=1}^N W_{ij}^{(n)} \Delta + \Delta - \tau_c \theta_i^{(n)}. \end{aligned} \quad (4)$$

Essentially, this method establishes an exact mapping with a shift-ReLU ANNs in the linear regime of the shift-ReLU. Based on R-TTFS and its variants, we can convert each component of LLMs into TTFS SNNs. The **four** main components are the embedding layer, LayerNorm, attention mechanism, and dropout.

4.2 Component-level Encoding of LLMs via TTFS

4.2.1 TTFS Embedding layers

In contrast to conventional embedding retrieval via table lookup, the TTFS embedding layer computes representations through sparse spike-timing-based multiplication. Specifically, input token IDs or positions are one-hot encoded, where an active entry (1) corresponds to a spike emitted at a user-defined $1ms$, while an inactive entry (0) implies silence (Figure 3, Left). Given these input timings, the output spike timings of the embedding layer are derived according to Equation 3.

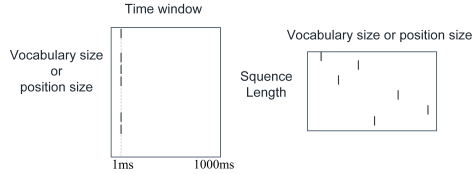


Figure 3: Left. A set of neurons representing the vocabulary or positions are waiting to be activated by input tokens. The time window is set to $1000ms$. A spike is generated if the corresponding token ID or position exists. The spike timings are set to $1ms$. Right. Each token or position in the input sequence activates the corresponding neuron.

4.2.2 TTFS layer normalization

LayerNorm consists of a sequence of elementary operations, including mean and variance computation, normalization, and affine transformation. In the TTFS implementation, all these operations are implemented within TTFS layers whose weights are fixed throughout LLMs training. Accordingly, we unroll Equation 5 into the seven steps shown in Figure 4.

$$\text{LN}(x_i) = \gamma \cdot \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (5)$$

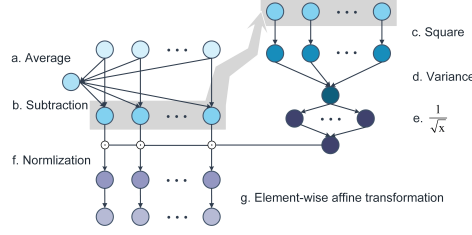


Figure 4: Integration for LayerNorm.

The mean TTFS neuron model. To compute the mean in TTFS, the neuron must aggregate all input spikes over the entire time window and encode the resulting mean value as a spike time. We denote this output spike time as t_μ . Since this aggregation requires observing all inputs within the window, the result necessarily lags behind the actual input events. Therefore, the output spike cannot be emitted within the current window and is instead assigned to the subsequent time bin. The dynamics of the mean neuron also follows the Equation 3. To enable averaging using SNNs with TTFS, it is essential to ensure that the network has fixed parameters:

$$\frac{1}{N} \stackrel{\text{def}}{=} W_{ij}, 0 \stackrel{\text{def}}{=} \sum_{j=1}^N W_{ij} \Delta + \Delta - \tau_c \theta_i. \quad (6)$$

The subtraction TTFS neuron model. The subtraction TTFS neuron has only two input spike timings, one is t_μ spanning $(t_{\min}^{(n)}, t_{\max}^{(n)}]$, the other one is the corresponding input spike timings t_j which lies in $(t_{\min}^{(n-1)}, t_{\max}^{(n-1)}]$. The dynamics of the subtraction neuron follows :

$$\tau_c \frac{dV_i^{(n+1)}}{dt} = \begin{cases} W_j H(t - t_j^{(n-1)}) - W_\mu H(t - t_\mu^{(n)}), & t < t_{\min}^{(n+1)} \\ 1, & t_{\min}^{(n+1)} \leq t \leq t_{\max}^{(n+1)} \end{cases} \quad (7)$$

We encode the difference between each input and t_μ as a spike timing t_{sub} . When the parameters satisfy Equation 9 where $W_j, W_\mu \in \mathbb{R}$ are scalars, Equation 8 can simulate subtraction effectively.

$$t_{ref}^{(n+1)} - t_{sub}^{(n+1)} = W_j(t_{ref}^{(n-1)} - t_j^{(n-1)}) - W_\mu(t_{ref}^{(n)} - t_\mu^{(n)}) + (3W_j + W_\mu + 1)\Delta - \tau_c \theta_i^{(n+1)} \quad (8)$$

$$1 \stackrel{\text{def}}{=} W_j, 1 \stackrel{\text{def}}{=} W_\mu, 0 \stackrel{\text{def}}{=} (3W_j + W_\mu + 1)\Delta - \tau_c \theta_i^{(n+1)}. \quad (9)$$

The square TTFS neuron model. Each square TTFS neuron receives input from only one presynaptic neuron. Suppose the input t_j belongs to $(t_{\min}^{(n-1)}, t_{\max}^{(n-1)}]$. The dynamics of the square neuron is defined by Equation 10 where W is a scalar.

$$\tau_c \frac{dV_i^{(n)}}{dt} = \begin{cases} WH(t - t_j^{(n-1)})(2t_j^{(n-1)} - t_{\min}^{(n-1)} - t_{\max}^{(n-1)}), & t < t_{\min}^{(n)} \\ 1, & t_{\min}^{(n)} \leq t \leq t_{\max}^{(n)} \end{cases} \quad (10)$$

Upon integrating Equation 10, the square operation can be exactly mapped to a TTFS layer (Equation 11). When the parameters satisfy Equation 12, the input x and the output y are equal to $t_{ref}^{(n-1)} - t_{sub}$ and $t_{ref}^{(n)} - t_{sq}$, respectively.

$$t_{ref}^{(n)} - t_i^{(n)} = W(t_{ref}^{(n-1)} - t_j^{(n-1)})^2 + W\Delta^2 + \Delta - \tau_c \theta \quad (11)$$

$$1 \stackrel{\text{def}}{=} W, 0 \stackrel{\text{def}}{=} W\Delta^2 + \Delta - \tau_c \theta. \quad (12)$$

The variance TTFS neuron model. The variance is computed as the mean of all squared inputs; therefore, the process follows the same principle as the mean TTFS neuron.

A TTFS-based approximation of $\frac{1}{\sqrt{x}}$.

The nonlinear operations—particularly division and square root extraction—pose significant challenges for representation through neuronal dynamics’ integral functions, and thus must be approximated.

Following the universal group operator Jiang et al. [2024], we use two fully connected TTFS layers to approximate this non-linearity. The first layer follows the $B1$ -model as defined in Equation 2, and its outputs are encoded as spike timings. The second layer produces membrane potentials and consists of non-spiking readout neurons. The dynamics of the output neurons are modeled in Equation 13, and the membrane potential is given by Equation 14 Stanojevic et al. [2024].

$$\tau_c \frac{dV_i^{(n)}}{dt} = \sum_j W_{ij}^{(n)} H(t - t_j^{(n-1)}) \quad (13)$$

$$V_i^{(n)} = \sum_j W_{ij}^{(n)} (t_{ref}^{(n-1)} - t_j^{(n-1)}) + \sum_j W_{ij}^{(n)} \Delta + D \quad (14)$$

The loss is defined as the mean squared error (MSE) between the outputs of $\frac{1}{\sqrt{x}}$ and the predictions. The fitting results are illustrated in Figure 5. During TTFS model training, we use the well-trained weights to initialize the TTFS-based approximation $\frac{1}{\sqrt{x}}$ and keep them fixed. **The normalization**

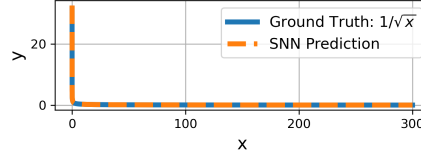


Figure 5: Fitting $\frac{1}{\sqrt{x}}$ by using two fully connected layer.

TTFS neuron model. To simulate $\frac{x_i - \mu}{\sigma}$, the input spike timing $t_{sub} \in (t_{min}^{n-1}, t_{max}^n]$ is multiplied by the weights (W_σ). When $(W_\sigma + 1)\Delta - \tau_c\theta$ is equal to 0, Equation 15 can simulate $\frac{x_i - \mu}{\sigma}$.

$$t_{ref}^{(n)} - t_{norm}^{(n)} = W_\sigma (t_{ref}^{(n-1)} - t_{sub}^{(n-1)}) + (W_\sigma + 1)\Delta - \tau_c\theta \quad (15)$$

Element-wise affine transformation TTFS neuron model. The operation corresponds to the Hadamard product between the spike timings $t_{norm}^{(n)}$ and the scaling parameter γ , and then add β , which means there is only one input spike in this TTFS neuron. When W_σ is equal to γ and $(W_\sigma + 1)\Delta - \tau_c\theta$ is equal to β , Equation 16 be mapped to the standard LayerNorm formulation.

$$t_{ref}^{(n)} - t_{LN}^{(n)} = W \odot (t_{ref}^{(n-1)} - t_{norm}^{(n-1)}) + (W + 1)\Delta - \tau_c\theta \quad (16)$$

4.2.3 TTFS Attention Block

Attention score computation. Q and K are represented by membrane potential $V_Q^{(n)}$ and spike timing $t_K^{(n)}$, respectively.

$$V_Q^{(n)} = \sum_j W_{qj}^{(n)} (t_{ref}^{(n-1)} - t_j^{(n-1)}) + \sum_j W_{qj}^{(n)} \Delta + D_q \quad (17)$$

$$t_{ref}^{(n)} - t_K^{(n)} = \sum_j W_{kj}^{(n)} (t_{ref}^{(n-1)} - t_j^{(n-1)}) \quad (18)$$

By treating $V_Q^{(n)}$ as the weights, the equation $V_{QK}^{(n)} = t_K^{(n)} V_Q^{(n)}$ can be reformulated as:

$$\tau_c \frac{dV_{QK}^{(n)}}{dt} = \sum_j V_Q^{(n)} H(t - t_K^{(n)}) \quad (19)$$

Matrix multiplication on neuromorphic hardware is replaced by the accumulation of spike inputs (K) weighted by linear synaptic kernels (Q).

Table 1: Performance comparison on the GLUE benchmark. F1 scores are reported for QQP and MRPC, Spearman correlations are reported for STS-B, and accuracy scores are reported for the other tasks.

Model	Size	MNLI-(m/mm)	QQP	QNLI	SST-2	STS-B	MRPC	RTE	Avg.
BERT	Base	84.0/79.6	87.4	86.2	92.5	88.8	85.2	68.6	84.0
SpikeBERT		71.4/71.0	68.2	66.4	85.4	18.7	82.0	57.5	65.1
SpikingBERT		78.1/-	86.8	85.2	88.2	81.9	79.2	66.1	-
SpikeLM		77.1/77.2	83.9	85.3	87.0	84.9	85.7	69.0	81.3
BPSN		74.1/-	83.9	83.9	88.0	83.9	85.3	64.6	-
TTFS-BERT		77.1/77.6	87.0	85.2	86.3	82.0	81.6	66.8	80.5
BERT	Large	82.1 /82.6	87.3	89.8	90.9	86.3	89.7	73.4	85.3
TTFS-BERT		81.6/80.9	86.3	85.6	90.1	82.6	84.0	67.4	82.3

Softmax. Given that the membrane potential encodes the QK product, the softmax operation can be performed directly.

Attention aggregation. Similarly, V can be expressed as Equation 20 and attention aggregation can be treated as Equation 21.

$$t_{\text{ref}}^{(n)} - t_V^{(n)} = \sum_{j=1}^N W_{vj}^{(n)} (t_{\text{ref}}^{(n-1)} - t_j^{(n-1)}) \quad (20)$$

$$t_{\text{ref}}^{(n+1)} - t_{\text{attn}}^{(n+1)} = V_{QK}^{(n)} (t_{\text{ref}}^{(n)} - t_V^{(n)}) + (V_{QK}^{(n)} + 1)\Delta - \tau_c \theta \quad (21)$$

4.2.4 The TTFS-based Dropout

The TTFS-based dropout follows the same principle as Equation 3, with the key difference being that it randomly drops certain neurons according to a specified dropout rate.

5 Experiments

5.1 Experimental Setup

We validate the effectiveness of our TTFS method by applying it to two representative architectures. For the encoder-based setting, we pre-train TTFS-BERT-base (110M parameters) and TTFS-BERT-Large (340M) Devlin et al. [2019] on Wikipedia and BookCorpus Zhu et al. [2015] to do next sentence prediction (NSP) and masked language model (MLM) Devlin et al. [2019] tasks. These models are subsequently fine-tuned on the GLUE benchmark Wang et al. [2018] and benchmarked against BERT Devlin et al. [2019], SpikeBERT Lv et al. [2023], BPSN Su et al. [2024], SpikingBERT Bal and Sengupta [2024], and SpikeLM Xing et al. [2024a]. Additionally, for the decoder-based setting, we pre-train TTFS-GPT-2 Small (117M) and TTFS-GPT-2 XL (1.5B) Radford et al. [2019] using the FineWeb-Edu dataset Penedo et al. [2024]. We assess their generation capabilities by reporting perplexity (PPL) and accuracy metrics via the lm-evaluation-harness Gao et al. [2021]. Due to the scarcity of full-scale Spiking GPT-2 models, we benchmark our approach against vanilla GPT-2 as the standard baseline, while simultaneously extending our comparison to recent RWKV-based spiking decoders, such as NSLLM Xu et al. [2026] and SpikeGPT Zhu et al. [2023]. Detailed experimental settings are provided in Appendix B.

5.2 Experimental Results

TTFS-BERT. In Table 1, we report the GLUE classification performance of models with different sizes. On the Base-sized model, TTFS-BERT achieves an average score of 80.5, significantly outperforming early SNN conversion methods like SpikeBERT (65.1) by a large margin. It also performs comparably to the leading SNN method, SpikeLM (81.3), while surpassing it on specific tasks such as QQP (87.0 vs. 83.9). We further validate the scalability of our proposed method on the large architecture. TTFS-BERT (Large) attains an average score of 82.3, leaving a gap of

Table 2: Performance comparison of conventional and TTFS-based GPT2 models on language modeling and common-sense reasoning benchmarks. All evaluations are performed using lm-evaluation-harness.

Model	Size	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA acc ↑	Hella. acc_n ↑	Wino. acc ↑	ARC-e acc ↑	ARC-c acc_n ↑	SIQA acc ↑	BoolQ acc ↑	Avg.
GPT-2	Small	37.4	40.1	32.6	62.5	31.1	51.6	39.5	22.7	36.6	48.7	40.7
NSLLM		-	-	-	59.3	-	51.0	39.9	23.3	-	-	-
NSLLM*		-	-	-	60.5	-	50.8	38.3	22.8	-	-	-
TTFS-GPT-2		40.4	92.6	25.4	63.9	34.3	50.6	47.0	25.7	37.9	55.1	42.5
GPT-2	XL	20.4	10.6	51.2	70.5	50.9	58.3	51.1	28.5	40.3	61.8	51.6
NSLLM		-	-	-	66.6	-	50.8	45.3	26.8	-	-	-
NSLLM*		-	-	-	63.7	-	50.2	42.1	23.6	-	-	-
TTFS-GPT-2		26.6	23.8	39.4	69.4	44.5	51.1	57.7	31.1	40.6	62.0	49.5

3.0 points with respect to the full-precision BERT-Large (85.3). **TTFS-GPT2**. Table 2 present zero-shot accuracy on commonsense reasoning benchmarks and PPL results on language modeling tasks. Specifically, TTFS-GPT-2 (Small) outperforms the ANN baseline on common-sense reasoning benchmarks (Avg. 42.5 vs. 40.7), while the XL model maintains comparable performance (49.5 vs. 51.6). On language modeling, by contrast, the gap is substantial and we do not claim parity: WikiText perplexity degrades from 20.4 to 26.6 on XL, and LAMBADA degrades from 10.6 to 23.8 in perplexity and from 51.2 to 39.4 in accuracy. Discretising spike timings therefore does more than perturb the confidence of the output distribution: because LAMBADA accuracy is itself a rank-based metric, its drop shows that the ranking of logits is altered on tasks that hinge on a single long-range dependency, where timing-quantisation error accumulates over the whole context before the decisive token is predicted. Commonsense reasoning benchmarks, whose candidate completions are short and well separated in likelihood, are far less sensitive to this effect, which is why accuracy there is preserved while perplexity is not. Closing this language-modeling gap is, in our view, the main open problem for TTFS-based generative models. In comparison with other spiking neural networks, we aligned our benchmark with SpikeGPT, which solely evaluates perplexity on WikiText-103. Under this setting, TTFS-GPT-2 XL reaches 26.6, outperforming the 39.8 perplexity of SpikeGPT (216M parameters), while TTFS-GPT-2 Small reaches 40.4, i.e. on par with SpikeGPT using roughly half the parameters.

6 Discussion

6.1 Energy Consumption Analysis

It is oversimplistic to expect a definitive conclusion regarding the energy consumption of SNNs, as it is highly dependent on the underlying hardware. While SNNs are energy efficient on neuromorphic hardware, they often incur higher latency and energy costs during training or inference on standard GPUs. Furthermore, it is imprudent to compare the energy consumption of SNNs and ANNs based solely on the distinction between multiply-and-accumulate (MAC) and accumulate (AC) operations Xing et al. [2024b], Bal and Sengupta [2024], Hwang et al. [2024], Zhao et al. [2025], as this perspective neglects the time and energy costs associated with implementing the internal dynamics of spiking neurons Davies et al. [2021].

$$\sum_{i',n} (T_r + 0.5(\vartheta_{i'}^{(n)})^2 C) + 0.5 \sum_{i'',n} (V_{i''}^{(n)}(t_{\max}^{(n)}))^2 C \quad (22)$$

We did not run our models on physical neuromorphic hardware, so the analysis below quantifies only the spike-related component of energy under an established cost model, and should be read as a proxy rather than as a measurement. We used Equation 22 to estimate the dominant factor of energy consumption of neuromorphic hardware, which contains the transmission cost T_r per spike and the charging cost determined by the capacitor C Stanojevic et al. [2024]. A neuron incurs significantly higher energy costs when it spikes compared to when it remains silent ($T_r \gg 0.5(\vartheta_{i'}^{(n)})^2 C > 0.5 \sum_{i'',n} (V_{i''}^{(n)}(t_{\max}^{(n)}))^2 C$) Stanojevic et al. [2024]. This observation suggests that, under this cost

model, TTFS with at most one spike incurs a lower spike-related cost than rate coding methods. To further quantify this effect, we estimate the spike-related energy consumption of various models using $N_s \cdot T_r$ as the primary metric, where N_s denotes the total spike count. This proxy deliberately excludes weight storage and off-core memory traffic, inter-core communication, and the cost of the timing circuitry needed to resolve δt . On digital asynchronous hardware these terms can dominate and do not scale with spike count, so Table 3 compares spike activity between spiking models and does not establish a system-level energy advantage over ANNs. Validating these estimates on a physical chip remains future work.

Table 3: Estimated average spike-related energy cost per neuron per inference. Baseline entries are obtained as (firing rate) $\times$ (number of time steps) from the settings reported in the respective papers: SpikeLM 0.3×4 and SpikingBERT 0.2×16 . The TTFS-BERT entry is derived from Table 4 in Appendix B.

	SpikeLM	SpikingBERT	TTFS-BERT
Spike-related cost	$1.2T_r$	$3.2T_r$	$0.80T_r$

7 Conclusion

In this work, we propose a R-TTFS coding scheme tailored for LLMs, formulating TTFS layers to effectively represent or approximate the embedding layer, LayerNorm, attention mechanism, and dropout. By constraining neurons to emit at most one spike, our approach inherently guarantees high sparsity. Under a spike-count proxy, this translates into a lower spike-related cost than rate-coded spiking baselines, while the model stays competitive with its ANN counterpart on natural language understanding and commonsense reasoning. Two limitations remain open: a clear language-modeling gap, most visible on LAMBADA, and the absence of measurements on physical neuromorphic hardware, which is required before any system-level energy claim can be made. Overall, our work opens up a promising avenue toward low-energy inference in LLMs, bridging the gap between deep learning and neuromorphic computing.

References

- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Malyaban Bal and Abhronil Sengupta. Spikingbert: Distilling bert to train spiking language models using implicit differentiation. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 10998–11006, 2024.
- Mohammad Bavandpour, Mohammad Reza Mahmoodi, and Dmitri B Strukov. Energy-efficient time-domain vector-by-matrix multiplier for neurocomputing and beyond. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 66(9):1512–1516, 2019.
- Mike Davies, Andreas Wild, Garrick Orchard, Yulia Sandamirskaya, Gabriel A Fonseca Guerra, Prasad Joshi, Philipp Plank, and Sumedh R Risbud. Advancing neuromorphic computing with loihi: A survey of results and outlook. *Proceedings of the IEEE*, 109(5):911–934, 2021.
- Brian DePasquale, David Sussillo, LF Abbott, and Mark M Churchland. The centrality of population-level factors to network computation is demonstrated by a versatile approach for training spiking networks. *Neuron*, 111(5):631–649, 2023.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- Zachary Friedenberger, Emerson Harkin, Katalin Tóth, and Richard Naud. Silences, spikes and bursts: Three-part knot of the neural code. *The Journal of Physiology*, 601(23):5165–5193, 2023.

- Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff, et al. A framework for few-shot language model evaluation. *Zenodo*, 2021.
- Nikhil Garg, Davide Florini, Patrick Dufour, Eloir Muhr, Mathieu C Faye, Marc Bocquet, Damien Querlioz, Yann Beilliard, Dominique Drouin, Fabien Alibart, et al. Versatile cmos analog lif neuron for memristor-integrated neuromorphic circuits. In *2024 International Conference on Neuromorphic Systems (ICONS)*, pages 185–192. IEEE, 2024.
- Wulfram Gerstner and Werner M Kistler. *Spiking neuron models: Single neurons, populations, plasticity*. Cambridge university press, 2002.
- Wulfram Gerstner, Werner M Kistler, Richard Naud, and Liam Paninski. *Neuronal dynamics: From single neurons to networks and models of cognition*. Cambridge University Press, 2014.
- Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.
- Julian Göltz, Laura Kriener, Andreas Baumbach, Sebastian Billaudelle, Oliver Breitwieser, Benjamin Cramer, Dominik Dold, Akos Ferenc Kungl, Walter Senn, Johannes Schemmel, et al. Fast and energy-efficient neuromorphic deep learning with first-spike times. *Nature machine intelligence*, 3(9):823–835, 2021.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.
- Sangwoo Hwang, Seunghyun Lee, Dahoon Park, Donghun Lee, and Jaeha Kung. Spikedattention: Training-free and fully spike-driven transformer-to-snn conversion with winner-oriented spike shift for softmax operation. *Advances in Neural Information Processing Systems*, 37:67422–67445, 2024.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr, 2015.
- Yizhou Jiang, Kunlin Hu, Tianren Zhang, Haichuan Gao, Yuqian Liu, Ying Fang, and Feng Chen. Spatio-temporal approximation: A training-free snn conversion for transformers. In *The Twelfth International Conference on Learning Representations*, 2024.
- Anders Krogh and John Hertz. A simple weight decay can improve generalization. *Advances in neural information processing systems*, 4, 1991.
- Changze Lv, Tianlong Li, Jianhan Xu, Chenxi Gu, Zixuan Ling, Cenyuan Zhang, Xiaoqing Zheng, and Xuanjing Huang. Spikebert: A language spikformer learned from bert with knowledge distillation. *arXiv preprint arXiv:2308.15122*, 2023.
- Paul A Merolla, John V Arthur, Rodrigo Alvarez-Icaza, Andrew S Cassidy, Jun Sawada, Filipp Akopyan, Bryan L Jackson, Nabil Imam, Chen Guo, Yutaka Nakamura, et al. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*, 345(6197):668–673, 2014.
- Hesham Mostafa. Supervised learning based on temporal coding in spiking neural networks. *IEEE transactions on neural networks and learning systems*, 29(7):3227–3235, 2017.
- Christian Pehle, Sebastian Billaudelle, Benjamin Cramer, Jakob Kaiser, Korbinian Schreiber, Yannik Stradmann, Johannes Weis, Aron Leibfried, Eric Müller, and Johannes Schemmel. The brainscales-2 accelerated neuromorphic system with hybrid plasticity. *Frontiers in Neuroscience*, Volume 16 - 2022, 2022.
- Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Colin A Raffel, Leandro Von Werra, Thomas Wolf, et al. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in Neural Information Processing Systems*, 37:30811–30849, 2024.

- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- Bodo Rueckauer and Shih-Chii Liu. Conversion of analog to spiking neural networks using sparse temporal coding. In *2018 IEEE international symposium on circuits and systems (ISCAS)*, pages 1–5. IEEE, 2018.
- Shuaijie Shen, Chao Wang, Renzhuo Huang, Yan Zhong, Qinghai Guo, Zhichao Lu, Jianguo Zhang, and Luziwei Leng. Spikingssms: Learning long sequences with sparse and parallel spiking state space models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 20380–20388, 2025.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.
- Ana Stanojevic, Stanisław Woźniak, Guillaume Bellec, Giovanni Cherubini, Angeliki Pantazi, and Wulfram Gerstner. An exact mapping from relu networks to spiking neural networks. *Neural Networks*, 168:74–88, 2023.
- Ana Stanojevic, Stanisław Woźniak, Guillaume Bellec, Giovanni Cherubini, Angeliki Pantazi, and Wulfram Gerstner. High-performance deep spiking neural networks with 0.3 spikes per neuron. *Nature Communications*, 15(1):6793, 2024.
- Michael Stuck, Xingyun Wang, and Richard Naud. A burst-dependent algorithm for neuromorphic on-chip learning of spiking neural networks. *Neuromorphic Computing and Engineering*, 5(1):014010, 2025.
- Qiaoyi Su, Shijie Mei, Xingrun Xing, Man Yao, Jiajun Zhang, Bo Xu, and Guoqi Li. Snn-bert: Training-efficient spiking neural networks for energy-efficient bert. *Neural Networks*, 180:106630, 2024.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461*, 2018.
- Wenjie Wei, Malu Zhang, Hong Qu, Ammar Belatreche, Jian Zhang, and Hong Chen. Temporal-coded spiking neural networks with dynamic firing threshold: Learning with event-driven backpropagation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10552–10562, 2023.
- Sandro Widmer, Marcel Kossel, Giovanni Cherubini, Stanisław Woźniak, Pier Andrea Francese, Ana Stanojevic, Matthias Brändli, Klaus Frick, and Angeliki Pantazi. Design of time-encoded spiking neural networks in 7-nm cmos technology. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 70(9):3639–3643, 2023.
- Xingrun Xing, Boyan Gao, Zheng Zhang, David A Clifton, Shitao Xiao, Li Du, Guoqi Li, and Jiajun Zhang. Spikellm: Scaling up spiking neural network to large language models via saliency-based spiking. *arXiv preprint arXiv:2407.04752*, 2024a.
- Xingrun Xing, Zheng Zhang, Ziyi Ni, Shitao Xiao, Yiming Ju, Siqi Fan, Yequan Wang, Jiajun Zhang, and Guoqi Li. SpikeLM: Towards general spike-driven language modeling via elastic bi-spiking mechanisms. In *Forty-first International Conference on Machine Learning*, 2024b.
- Han Xu, Xuerui Qiu, Yunhui Xu, Mohammed E Elbittity, Peng Zhou, Yang Tian, Rui-Jie Zhu, Jiahong Zhang, Shaowei Gu, Yuqi Pan, et al. Neuromorphic spike-based large language model. *National Science Review*, 13(4):nwaf551, 2026.
- Man Yao, Guangshe Zhao, Hengyu Zhang, Yifan Hu, Lei Deng, Yonghong Tian, Bo Xu, and Guoqi Li. Attention spiking neural networks. *IEEE transactions on pattern analysis and machine intelligence*, 45(8):9393–9410, 2023.

Malu Zhang, Shuai Wang, Jibin Wu, Wenjie Wei, Dehao Zhang, Zijian Zhou, Siying Wang, Fan Zhang, and Yang Yang. Toward energy-efficient spike-based deep reinforcement learning with temporal coding. *IEEE Computational Intelligence Magazine*, 20(2):45–57, 2025.

Lusen Zhao, Zihan Huang, Jianhao Ding, and Zhaofei Yu. Ttfsformer: A ttfs-based lossless conversion of spiking transformer. In *Forty-second International Conference on Machine Learning*, 2025.

Zhaokun Zhou, Yuesheng Zhu, Chao He, Yaowei Wang, Shuicheng YAN, Yonghong Tian, and Li Yuan. Spikformer: When spiking neural network meets transformer. In *The Eleventh International Conference on Learning Representations*, 2023.

Rui-Jie Zhu, Qihang Zhao, Guoqi Li, and Jason K Eshraghian. Spikept: Generative pre-trained language model with spiking neural networks. *arXiv preprint arXiv:2302.13939*, 2023.

Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *Proceedings of the IEEE international conference on computer vision*, pages 19–27, 2015.

A Technical appendices and supplementary material

A.0.1 Theoretical implementation on neuromorphic chips

There is a slight difference between reference-based and conventional TTFS SNNs implemented on neuromorphic hardware. In contrast to the first branch of eq. (1), the reference-based neuron requires a gating signal to modulate the sign of integration as shown in fig. 2C. This ensures that negative inputs ($t_{ref} - t_j \leq 0$) can be represented correctly as integration proceeds chronologically.

$$\begin{cases} \int_{t_j}^{t_{ref}^{(n-1)}} W_{ij}^{(n)} H(t - t_j^{(n-1)}) dt, & t_{min}^{(n-1)} \leq t < t_{ref}^{(n-1)} \\ - \int_{t_{ref}^{(n-1)}}^{t_j} W_{ij}^{(n)} H(t - t_j^{(n-1)}) dt, & t_{ref}^{(n-1)} \leq t \leq t_{max}^{(n-1)} \end{cases} \quad (23)$$

B Implementation details

All pre-training experiments were conducted on a cluster of eight NVIDIA H100 GPUs (80GB memory each).

B.1 TTFS-BERT

We train BERT for 1,000,000 update steps using the AdamW optimizer with a peak learning rate of 1×10^{-4} . We employ a cosine learning-rate schedule with a 2% warm-up ratio, after which the learning rate is annealed to the end of training. For BERT models, we use the standard WordPiece tokenizer (vocabulary size 30,522). For pretraining, we set the maximum input sequence length to 512 tokens and the batch size is 512.

Derivation of the spike-count estimate. Summing the third column of Table 4, one TTFS transformer block contains $22d + N + 1028$ neurons; the constant term is $2 \times (512 + 2)$ because each of the two LayerNorm sub-blocks contains a 512-neuron $1/\sqrt{x}$ approximator together with its mean and variance neurons. Weighting each row by its average spike count gives $18.28d + N + 260$ spikes per block, where the non-integer contributions come from the $1/\sqrt{x}$ block (512×0.25 spikes per LayerNorm) and from TTFS_ReLU ($4d \times 0.07$); every remaining sub-block emits exactly one spike per neuron by construction. For TTFS-BERT-Base ($d = 768$, $N = 512$) this gives 14,811 spikes over 18,436 neurons, i.e. ≈ 0.80 spikes per neuron per inference, which is the $0.80 T_r$ entry of Table 3. Two remarks are worth making. First, the expanded TTFS LayerNorm does not increase the total spike count relative to a rate-coded block: a rate-coded block of $\sim 16d + N$ neurons firing at 1.2 spikes/neuron emits $\approx 19.2d + 1.2N$ spikes, which exceeds $18.28d + N + 260$ for the hidden sizes used here and grows faster in both d and N . Second, the $1/\sqrt{x}$ approximator contributes a constant 512 neurons per LayerNorm independently of d , so its relative cost vanishes as models are scaled up.

Table 4: Average Spike Count across Transformer Blocks per Time Window

Transformer Block	Sub-Block	Num. Neurons	Avg. Spike Count per Neuron per Time Window
TTFS_LayerNorm	mean	1	1
	subtraction	d	1
	square	d	1
	variance	1	1
	$\frac{1}{\sqrt{x}}$	512	0.25
	norm	d	1
	affine	d	1
TTFS_Attention	t_q	d	1
	t_k	d	1
	t_v	d	1
	t_{qk}	N	1
	t_{qkv}	d	1
TTFS_linear		d	1
TTFS_dropout		d	1
TTFS_res		d	1
TTFS_ReLU		$4d$	0.07
TTFS_linear		d	1
TTFS_dropout		d	1
TTFS_res		d	1
TTFS_LayerNorm	(Same as initial TTFS_LayerNorm)		1

Evaluation Metrics. We report specific metrics for each dataset. Accuracy measures the simple proportion of correct predictions. For tasks sensitive to class imbalance (QQP, MRPC), we utilize the F1 score, defined as the harmonic mean of precision and recall. Finally, for semantic similarity regression (STS-B), we employ Spearman correlation to assess the strength of the monotonic rank relationship between the model’s scores and human judgments.

B.2 TTFS-GPT-2

We train the model for 1,000,000 update steps with a peak learning rate of 1×10^{-4} and a batch size of 512. The learning rate follows a linear decay schedule with a 2% warm-up ratio. For GPT-2 models, we use the GPT-2 byte-level BPE tokenizer (vocabulary size 50,257). For GPT pretraining, we use a context window (block size) of 512 tokens.

Evaluation Metrics. Perplexity (PPL) and Accuracy (Acc) serve as complementary metrics to evaluate distinct dimensions of model performance. Perplexity acts as an intrinsic indicator of generative fluency, measuring the model’s uncertainty in predicting the next token; lower values reflect a more robust grasp of the underlying language distribution. In contrast, Accuracy is employed for downstream tasks (such as multiple-choice questions) to assess reasoning and knowledge application. It typically quantifies the model’s ability to correctly rank candidate answers based on conditional likelihoods, ensuring that the model assigns the highest probability to the correct completion rather than merely generating text.

C Detailed Results

fig. 6 (left) and fig. 7 show the training loss curves for BERT-base and BERT-large, respectively. The accuracies of the NSP and MLM tasks are shown in Fig. 6 (middle and right). Compared to BERT, the TTFS model exhibits almost no performance degradation.

C.1 Impact of temporal configurations on representation and sparsity

One of the advantages of TTFS coding is sparsity. In contrast to other SNN-based language models that rely on multiple spikes per unit Lv et al. [2023], Bal and Sengupta [2024], Zhu et al. [2023], Xing

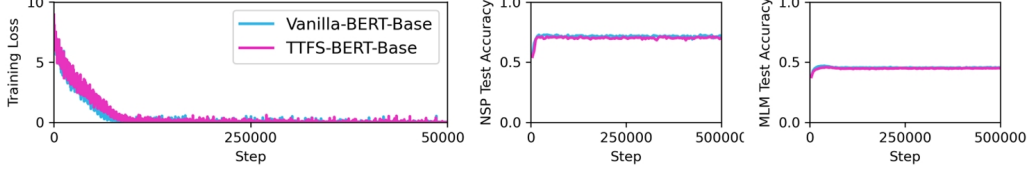


Figure 6: Comparison between Vanilla-BERT-Base and TTFS-BERT-Base.

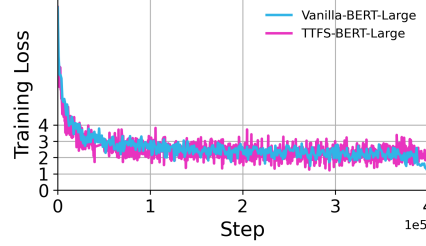


Figure 7: Comparison between Vanilla-BERT-Large and TTFS-BERT-Large.

et al. [2024b], Shen et al. [2025], our TTFS-LLMs use at most one spike per unit while remaining competitive on natural language understanding and commonsense reasoning.

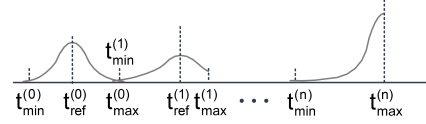


Figure 8: Schematic illustration of the distribution of first spike timings.

C.1.1 R-TTFS neuron VS B1-model

Figure 8 illustrates schematically the distribution of first-spike timings across layers. While the schematics for Layer 0 illustrate the timing distributions of symmetric R-TTFS neurons (which can be extended to an asymmetric setting in Layer 1), the schematic for Layer n depicts the distribution for B1-model neurons, which are restricted to representing outputs with a single sign.

C.1.2 Size of time window

One may argue that TTFS trades off sparsity for time, such that a neuromorphic TTFS device would be too slow. The TTFS time window ($T = t_{max} - t_{min}$) is user-defined, but must occur sequentially across layers (as shown in Figure 8) such that the total duration time duration is $T * M$ where M is the number of layers. The considerations in choosing the time window are, like in rate-coded SNNs, a trade-off between expressibility and total latency. The expressibility of a spiking unit can be measured by the number of distinct times that can be reliably distinguished, which relates to the bit precision of the floating point number represented by the unit. For TTFS, this is the per-unit duration divided by the smallest accessible spike-timing precision δt : $N_{states} = T/\delta t$. For a rate-coding unit, the number of states corresponds to the number of time bins which ultimately define the minimum and maximum number of spikes, and thus again $N_{states} = T/\delta t$. Thus TTFS does not bring a longer latency per unit.

The fact that TTFS is restricted to sequentially go through each layer one at a time can, however, give rise to longer latencies than rate-coding, which does not have this fundamental constraining. It should be noted that, while rate-coding does not require layer activity to be computed sequentially, letting the rate-coding units follow absolute time is not easy to implement. Most efforts that use SNNs in the context of ANNs utilize a layer-specific time, such that there would be no advantage on the total

latency Zhu et al. [2023], Yao et al. [2023], Zhou et al. [2023], Shen et al. [2025]. Some efforts have specifically attempted to have a network follow absolute time but scaling up to LLMs remains to be proven Stuck et al. [2025].

Another related consideration is that some inputs may fall into the saturated region of the TTFS mapping, producing nearly fixed spike timings and thus degrading expressivity. However, in practice, this issue hardly happens because weight initialization Glorot and Bengio [2010], He et al. [2015], normalization Ioffe and Szegedy [2015], Ba et al. [2016] and regularization Krogh and Hertz [1991], Srivastava et al. [2014] help keep the outputs of each hidden layer within a reasonable range. Among them, normalization explicitly centers the outputs around zero, while initialization and regularization mainly control the scale. As a result, it becomes straightforward to adjust $t_{\max}$ such that the output of the TTFS neurons aligns with the output of the corresponding layer in the ANNs.

C.1.3 Firing rate

We set the time window to 1000 ms and report the firing rate in kHz. We obtain the mean firing rates of the embedding layers, LayerNorm, dropout, and transformer blocks, as shown in Figure 9, where the firing rate is approximately 8.92×10^{-4} kHz. For reference, rate-based SNN-LLMs report values between 0.1 and 0.6 Su et al. [2024], Bal and Sengupta [2024], Xing et al. [2024b], although the original papers do not specify units.

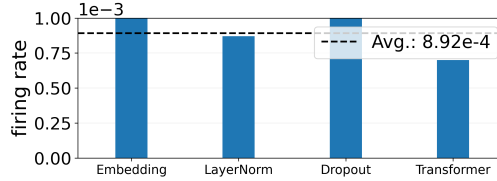


Figure 9: Firing rate in TTFS-BERT.